

Experimentação - Sprint 3

Testamos os quatro mecanismos de atenção implementados nessa sprint (Self-Attention, Scaled Dot-Product, Causal Attention e Multi-Head) em cima das sugestões que o professor passou. Deu pra encaixar tudo em 3 experimentos, reaproveitando as classes que já estavam prontas nos arquivos 3,1.py a 3,4.py.

Experimento 1 - Dimensão de embedding, número de heads e head_dim

Diferentes dimensões de embedding

Diferentes números de heads

Diferentes dimensões de head

O que testar: como d_out e num_heads mudam o head_dim e o shape final da saída.

Como fizemos: pegamos a **MultiHead Attention** do 3,4.py e rodamos a mesma frase de teste ("Trevizol e Pedro constroem um LLM do zero juntos") variando primeiro d_out (8, 16, 32) e depois num_heads (1, 2, 4, 8). No final forçamos um num_heads que não divide d_out, só pra ver o assert travar mesmo.

```
=== Variando dimensão de embedding (num_heads=2) ===
    d_in=d_out= 8 -> shape=(9, 8) head_dim=4
    d_in=d_out=16 -> shape=(9, 16) head_dim=8
    d_in=d_out=32 -> shape=(9, 32) head_dim=16

=== Variando número de heads (d_out=32 fixo) ===
    num_heads=1 -> head_dim=32 shape=(9, 32)
    num_heads=2 -> head_dim=16 shape=(9, 32)
    num_heads=4 -> head_dim= 8 shape=(9, 32)
    num_heads=8 -> head_dim= 4 shape=(9, 32)

    === num_heads=5 com d_out=32 ===
    AssertionError: d_out precisa ser divisível por num_heads
```

O tamanho do embedding não interfere na quantidade de tokens, só no vetor de contexto de cada um — o shape final é sempre (n_tokens, d_out). O número de heads já divide esse mesmo espaço: $\text{head_dim} = \text{d_out} / \text{num_heads}$. Mais heads = cada uma vendo um pedaço menor, mas processando em paralelo. E o erro no final confirma o óbvio: não dá pra dividir 32 em 5 partes iguais, então nem chega a rodar o cálculo.

Experimento 2 — Self-Attention x Multi-Head, e com/sem escala

- Comparação entre Self-Attention e Multi-Head Attention
- Comparação entre Attention com e sem escala

O que testar: se o vetor de contexto muda entre self-attention (1 cabeça) e multi-head (várias), e o efeito de dividir os scores por $\sqrt{d_k}$ antes do softmax.

Como fizemos: rodamos a mesma entrada no **Self Attention v2** (3,2.py) e no **MultiHead Attention** (3,4.py, 4 heads) e comparamos a saída. Depois, com $d_k=64$ (mais perto do que se usa de verdade), calculamos os pesos de atenção com e sem a escala, pra ver o quanto a distribuição muda — usamos entropia pra medir isso numericamente.

Diferença media (Self-Attention vs Multi-Head): 0.0453

SEM escala - maior peso: 0.2030 | entropia media: 2.1414

COM escala - maior peso: 0.1210 | entropia media: 2.1963

Entropia máxima possível (9 tokens): 2.1972

Self-Attention e Multi-Head deram vetores de contexto diferentes (~0.045 de diferença média), o que faz sentido, já que cada cabeça do Multi-Head tem seu próprio conjunto de pesos operando num espaço menor.

Sobre a escala: sem ela, a atenção ficou bem mais concentrada (maior peso 0.20, entropia 2.14); com ela, ficou quase uniforme entre as 9 palavras (maior peso 0.12, entropia 2.20 — bem perto do máximo teórico). Bate com o que o livro explica na pág. 69: sem escalar, embeddings maiores geram produtos escalares grandes demais, o que empurra o softmax pra uma região de gradiente quase zero e trava o treino. Daí o nome scaled dot-product attention.

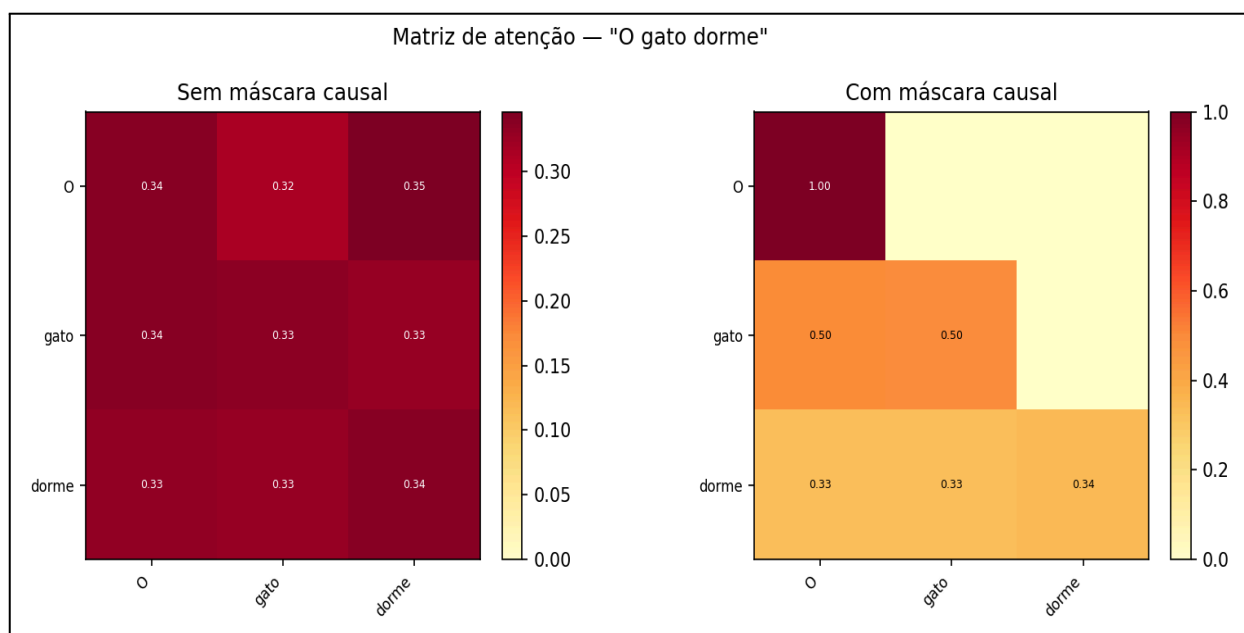
Experimento 3 — Matriz de atenção e máscara casual

- Visualização da matriz de atenção
- Comportamento da máscara causal
- Diferentes sequências de entrada

O que testar: como fica a matriz de atenção, visualmente, com e sem máscara causal, em frases de tamanhos diferentes.

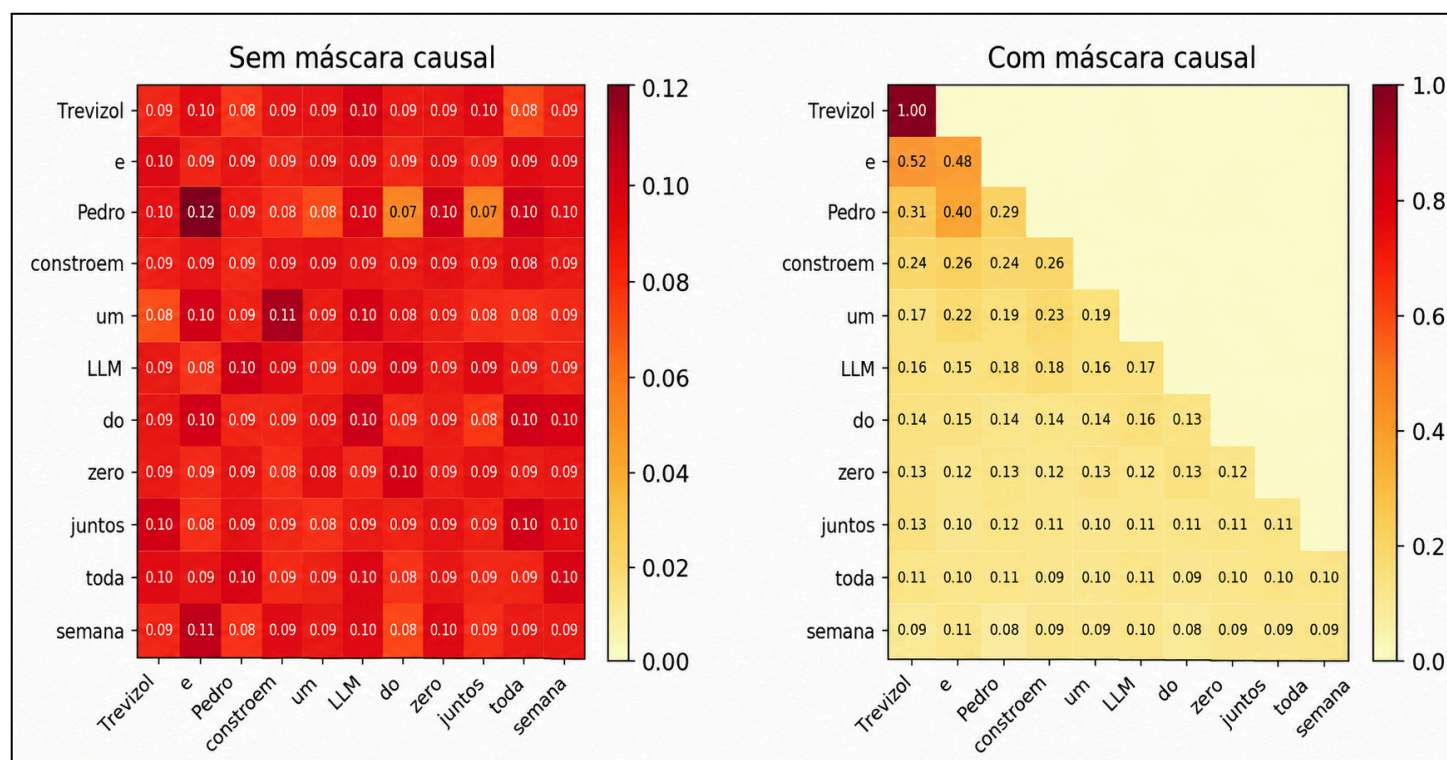
Como fizemos: calculamos a matriz de attention weights completa pra duas frases ("O gato dorme", 3 tokens, e uma de 11 tokens) e plotamos como heatmap, lado a lado, com e sem a máscara.

Frase curta: "O gato dorme"



Sem máscara, os pesos ficam parecidos entre si (~ 0.33). Com a máscara, dá pra ver o triângulo: "O" só atende a si mesma, "gato" divide entre "O" e ela mesma, e só "dorme" (última palavra) enxerga a frase inteira.

Frase longa (11 tokens)



Mesmo padrão numa frase maior: sem máscara a matriz inteira é preenchida, com máscara a metade de cima fica zerada. "Trevizol" (primeira palavra) só atende a si mesma; "semana" (última) é a única com a linha inteira preenchida.

Em resumo: independente do tamanho da frase, a máscara causal sempre gera uma matriz triangular, com cada linha ainda somando 1. É esse mecanismo que garante que o modelo nunca veja palavras futuras durante a geração — o que vamos plugar na arquitetura GPT completa da Sprint 4.